MINISTRY OF EDUCATION AND RESEARCH

TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 2

Software Design

Author: Iulia Maria Martin
Group: 30433

Faculty of Automation and Computer Science

March, 2026

Contents

0.1 Problem definition . . . 2

1 Analysis phase - Use cases & requirements . . . 2

1.1 Functional requirements . . . 2

1.2 Use cases . . . 2

2 Design Phase . . . 4

2.1 Architecture Overview . . . 4

2.2 Entity-Relationship Diagram . . . 5

2.3 Component X - Web Application Server . . . 6

2.4 Component Y - Database Layer . . . 6

2.5 Design Patterns . . . 7

2.6 Class Diagrams . . . 7

2.7 Sequence Diagrams . . . 9

3 Implementation Phase - Application . . . 10

3.1 Tools and Technologies . . . 10

3.2 Application Structure . . . 11

3.3 User Manual . . . 12

4 Testing and Validation . . . 13

0.1 Problem definition

This project extends the Flower Shop Management System developed in Assignment 1 by evolving it from a simple layered architecture into an event-driven, service-based architecture. The application continues to manage the complete workflow of a flower retail business while introducing new capabilities: domain event emission, email notifications, multi-format data export, and explicit implementation of additional design patterns. The system remains built on ASP.NET Core with Entity Framework Core and MySQL, but responsibilities are now clearly separated into modular services.

Development Phases

The project was extended in four phases: Phase 1 refactored the existing FlowerService to emit typed domain events on Create, Update, and Delete operations. Phase 2 introduced the NotificationService as an Observer that listens to these events and sends email notifications to the authenticated user. Phase 3 implemented the Strategy pattern for multi-format resource export (JSON, XML, CSV) with dynamic filtering and sorting support. Phase 4 integrated all new components into the existing MVC structure and verified role-based authorization across all controllers.

Additional Features and Deliverables

The system now includes an in-memory event bus implementing the Observer pattern, an SMTP-based email notification service, and a three-strategy export pipeline accessible directly from the UI. Design patterns are explicitly applied: Singleton for the database connection factory, Observer for event-driven communication, and Strategy for export format selection. All Assignment 1 features remain fully functional.

1 Analysis phase - Use cases & requirements

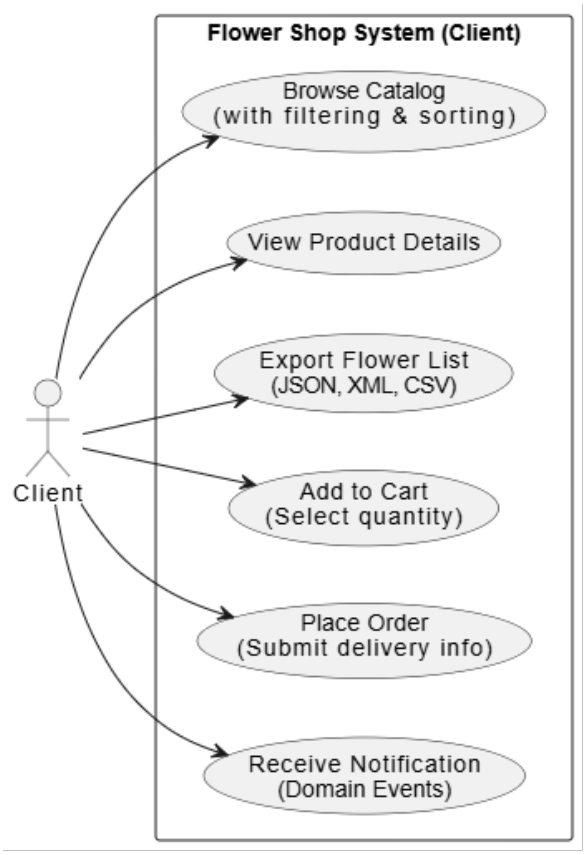
1.1 Functional requirements

The system fully implements all Assignment 1 functional specifications and extends them with event-driven and export capabilities. The FlowerService now publishes domain events after every successful CRUD operation, and the NotificationService subscribes to these events and delivers email notifications to the currently authenticated user. The export functionality allows Visitors, Clients, Florists, and Administrators to download a filtered and sorted list of flowers in JSON, XML, or CSV format at the press of a button. Role-based access control is enforced through authorization attributes on all controllers, with export filtering restricted in code for unauthenticated Visitors.

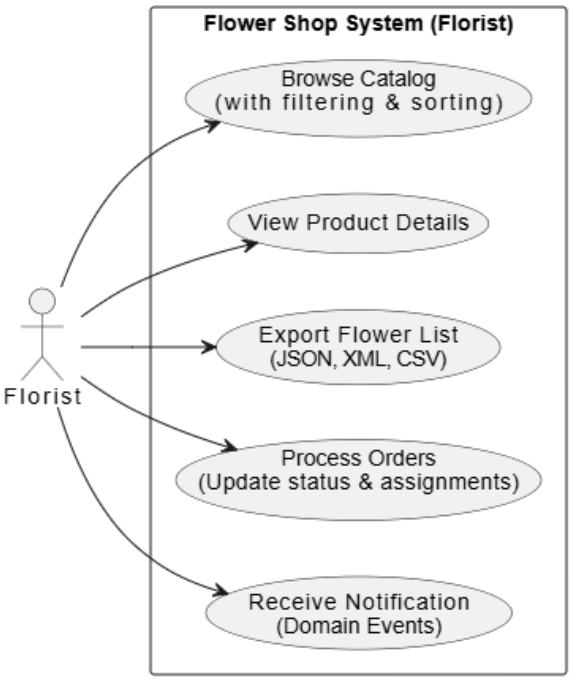
1.2 Use cases

Use cases describe in text the interactions between actors and the system in order to accomplish specific tasks.

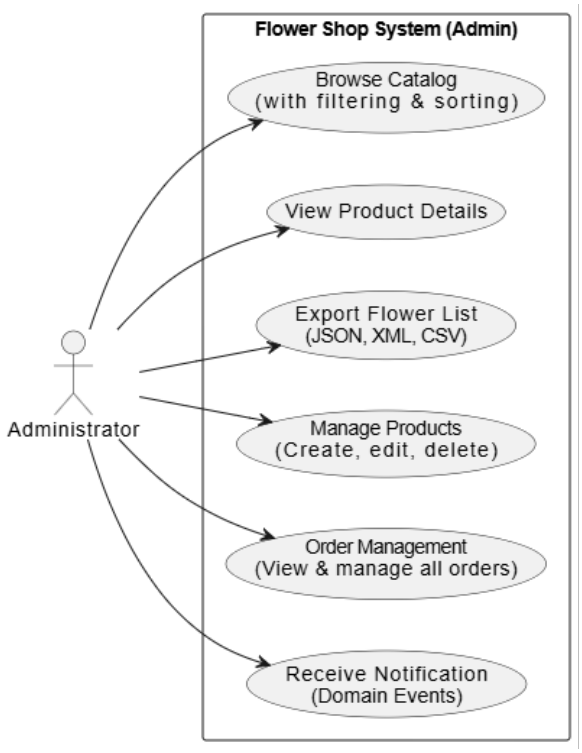
- **Browse Catalog** - Visitor/Client views flowers and bouquets with filtering and sorting
- **View Product Details** - Visitor/Client views detailed information about specific products
- **Export Flower List** - Any user downloads the flower catalog in JSON, XML, or CSV format
- **Add to Cart** - Client adds items to shopping cart with quantity selection
- **Place Order** - Client converts cart items into a formal order with delivery info
- **Manage Products** - Admin creates, edits, and deletes flowers and bouquets; events are emitted on each operation
- **Receive Notification** - Authenticated user receives an email after a flower is created, updated, or deleted
- **Process Orders** - Florist updates order status and manages assignments
- **User Authentication** - Visitor logs in to access personalized features
- **Order Management** - Admin views and manages all system orders



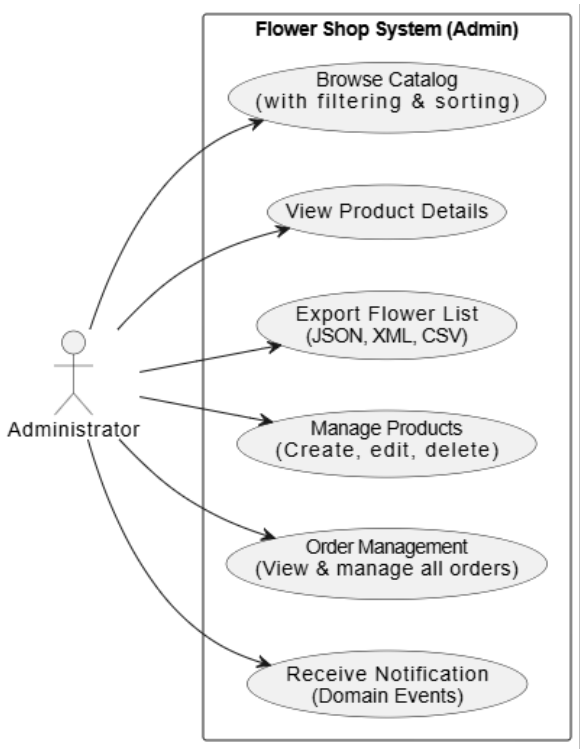
(a) Visitor Diagram



(b) Client Diagram



(a) Florist Diagram



(b) Admin Diagram

2 Design Phase

2.1 Architecture Overview

The application evolves from a simple layered architecture into an **event-driven service-based architecture**. Services remain part of the same application but are now organized into clearly separated modules with distinct responsibilities:

- **Resource Management Service** — FlowerService, BouquetService, CategoryService handle domain logic and now emit domain events after state changes
- **User Management Service** — UserService and authentication middleware manage authentication, session handling, and role resolution
- **Notification Service** — NotificationService subscribes to domain events via the event bus and delivers email notifications to the currently authenticated user
- **Export Service** — ExportController and ExportStrategyFactory coordinate multi-format data export using the Strategy pattern

Communication between modules is implemented through **event-driven communication** using the Observer pattern. The InMemoryEventBus acts as the Subject, maintaining a registry of event handlers. When FlowerService publishes an event, all subscribed handlers (Observers) are invoked synchronously. This decouples the resource service from the notification logic entirely.

This architectural evolution provides improved modularity and extensibility: new notification channels or export formats can be added without modifying existing services. The trade-off is increased indirection, which is managed through explicit interface contracts and XML documentation on all pattern participants.

2.2 Entity-Relationship Diagram

The domain model from Assignment 1 is unchanged. No new database tables are introduced in Assignment 2, as event handling and export are implemented entirely in-memory.

Users table: Stores user accounts with authentication data and role information. Important attributes include email (unique), password_hash (SHA256), and role (Client/Florist/Admin). The email field is used as the notification destination when domain events are triggered.

Florists table: Contains florist profiles with contact information. Links to Users table for account association and to Flowers table for product management.

Categories table: Organizes flowers into logical groups. Simple structure with name and description, has one-to-many relationship with Flowers.

Flowers table: Core product entity and primary event source. FlowerService emits FlowerCreatedEvent, FlowerUpdatedEvent, and FlowerDeletedEvent after operations on this table.

Bouquets table: Predefined flower arrangements with fixed pricing. Links to Orders and has many-to-many relationships with Flowers through the junction table.

Bouquet_Flowers table: Junction table implementing many-to-many relationships between Bouquets and Flowers with quantity information.

Orders table: Customer orders with status tracking and delivery information. Links to Users, optional Bouquets, and assigned Florists.

Order_Items table: Individual items within orders, supporting either flowers or bouquets. Maintains unit prices for historical accuracy.

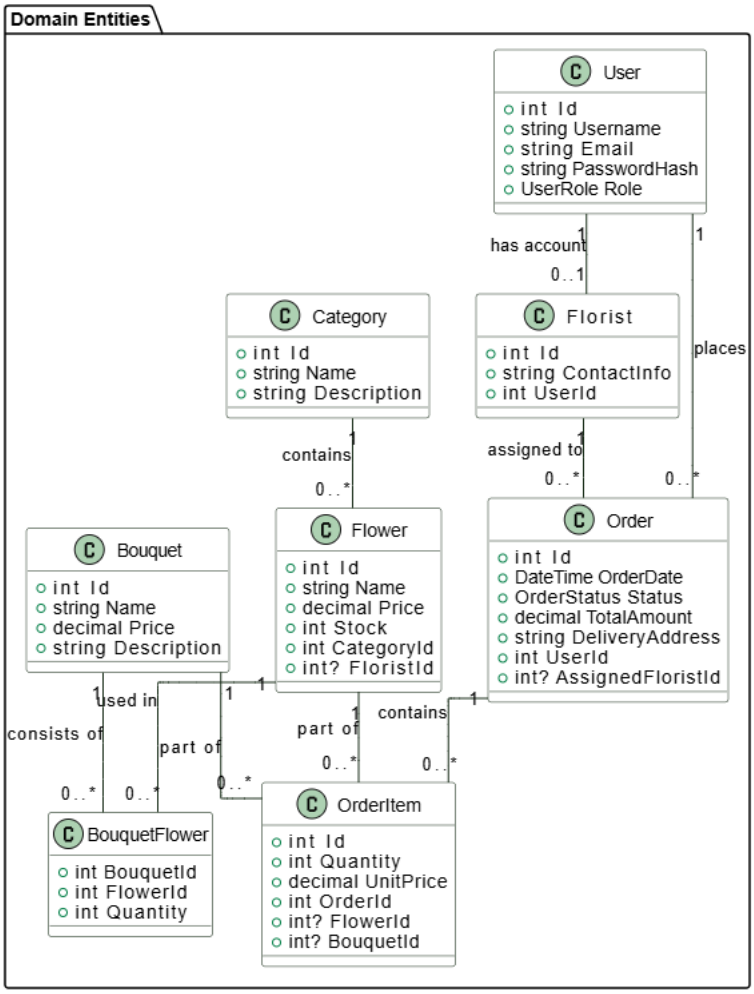


Figure 3: Entity-Relationship Diagram

2.3 Component X - Web Application Server

The web application server remains the main backend component built with ASP.NET Core. In Assignment 2 it is extended with three new responsibilities: event publication after resource mutations, email dispatch through the SMTP abstraction layer, and dynamic file generation for multi-format export. The server continues to handle HTTP requests, manage user sessions, coordinate business logic, and render HTML responses. Security is implemented through authentication middleware and role-based authorization attributes on all controllers.

Email notifications are dispatched through a Gmail account configured in appsettings.json, using an App Password generated via Google's two-step verification system. This allows the application to send real emails to any recipient address without exposing the account's primary password. The authenticated user's email address, stored in the session, is used as the notification recipient each time a flower is created, updated, or deleted.

2.4 Component Y - Database Layer

The database layer consists of a MySQL server with Entity Framework Core ORM. This component handles data persistence, query optimization, and transaction management. In As-

signment 2, the AppDbContext is additionally wrapped by an IDbContextFactory registered as a Singleton, making the Singleton pattern explicit in the DI configuration. The database supports concurrent connections and uses proper indexing for performance.

2.5 Design Patterns

The system implements the following design patterns to ensure code quality, maintainability, and extensibility.

- **Repository Pattern** (Structural): Unchanged from Assignment 1. Provides interface-based data access through IFlowerRepository, IBouquetRepository, IOrderRepository, and IUserRepository, each with concrete Entity Framework Core implementations.
- **Dependency Injection** (Creational): Unchanged from Assignment 1. All services, repositories, event handlers, and export strategies are registered in Program.cs and injected through constructors.
- **Service Layer Pattern** (Architectural): Unchanged from Assignment 1. Service interfaces and implementations coordinate between controllers and repositories.
- **Singleton** (Creational): The IDbContextFactory<AppDbContext> is registered as a Singleton in Program.cs, ensuring a single shared connection factory for the application lifetime. The InMemoryEventBus is also registered as a Singleton so that handler subscriptions made at startup persist for all subsequent requests.
- **Observer** (Behavioral): Implemented through IEventBus (Subject) and IEventHandler<T> (Observer). FlowerService publishes typed domain events (FlowerCreatedEvent, FlowerUpdatedEvent, FlowerDeletedEvent) after successful database operations. NotificationService subscribes to all three events at application startup and handles each by sending an email notification. This decouples the resource service from all notification concerns.
- **Strategy** (Behavioral): Implemented through IExportStrategy and three concrete strategies — JsonExportStrategy, XmlExportStrategy, and CsvExportStrategy. The ExportStrategyFactory selects the correct strategy at runtime based on the format parameter from the HTTP request. Adding a new format requires only a new class implementing IExportStrategy, with no changes to the controller or factory.
- **Factory** (Creational): ExportStrategyFactory receives all registered IExportStrategy implementations via DI and exposes a GetStrategy(string format) method, encapsulating the selection logic.

2.6 Class Diagrams

Class diagrams describe the static structure of the system by presenting classes, attributes, methods, and relationships between classes.

The event subsystem introduces IDomainEvent as a base interface carried by FlowerCreatedEvent, FlowerUpdatedEvent, and FlowerDeletedEvent, each holding entity metadata and the triggering user's email. IEventBus defines Subscribe and Publish methods, implemented by

InMemoryEventBus which maintains a dictionary mapping event types to handler lists. NotificationService implements IEventHandler for all three flower events and depends on IEmailSender, which is implemented by SmtpEmailSender.

The export subsystem introduces IExportStrategy with Format, ContentType, FileExtension, and Export<T> members. JsonExportStrategy, XmlExportStrategy, and CsvExportStrategy each implement this interface. ExportStrategyFactory holds a collection of all registered strategies and selects among them by format string. ExportController depends on ExportStrategyFactory and IFlowerService to retrieve, filter, sort, and serialize data.

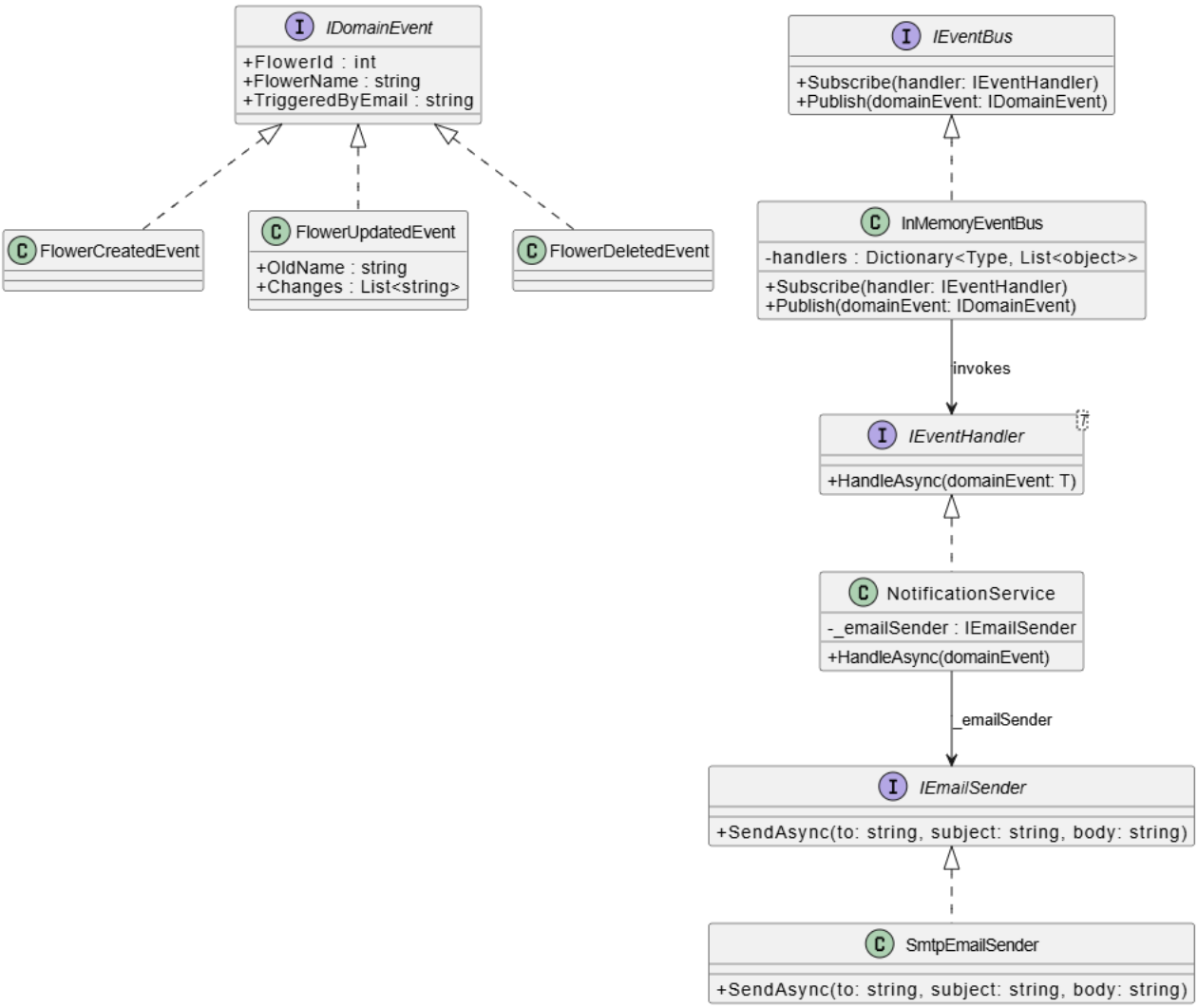


Figure 4: Class Diagram — Event

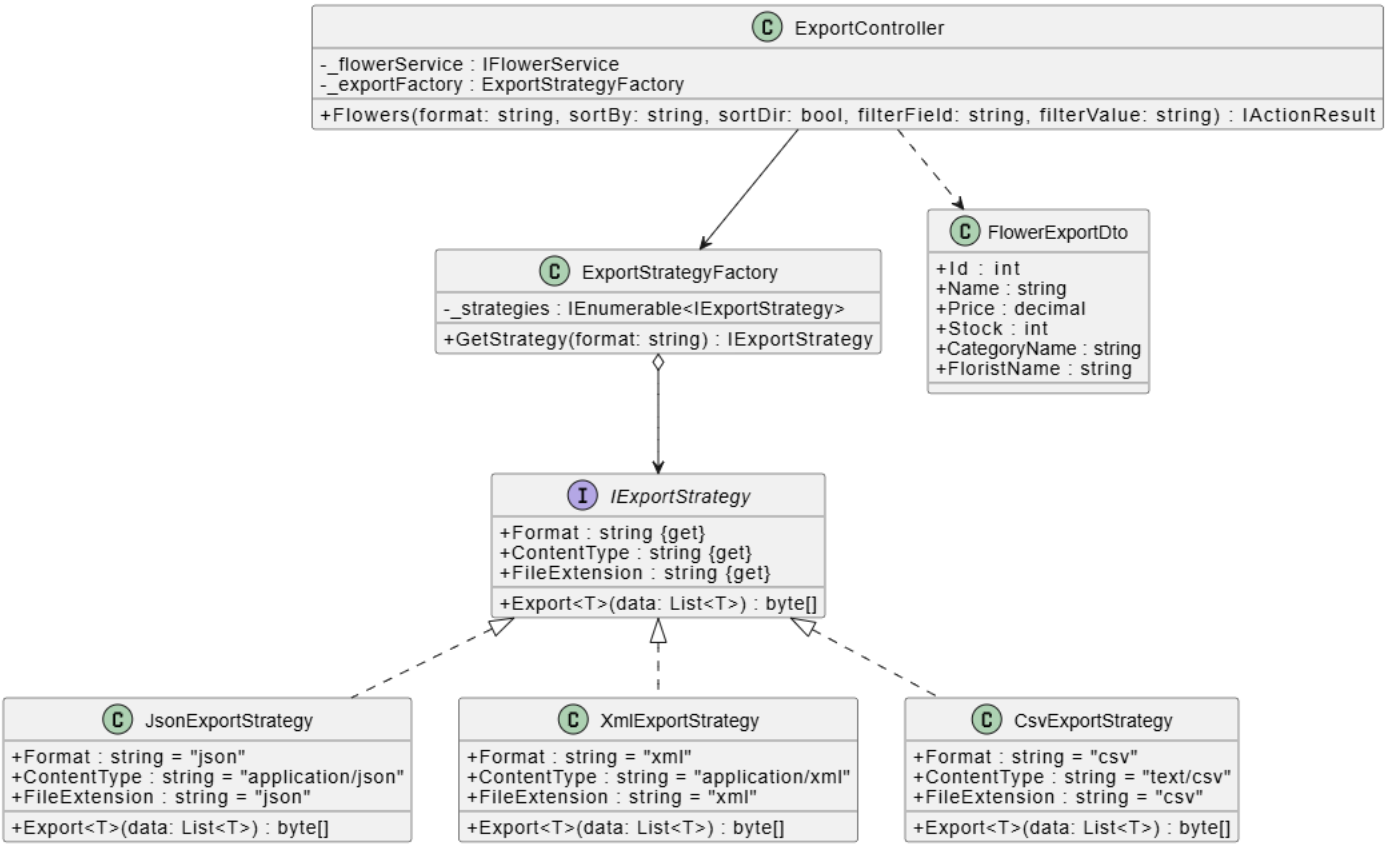


Figure 5: Class Diagram — Export Subsystems

2.7 Sequence Diagrams

Sequence diagrams illustrate how objects interact during the execution of a particular use case. They show the order of method calls and message exchanges between system components.

The flower creation sequence shows the extended workflow. The Admin submits the create form, FlowersController calls FlowerService.CreateAsync(), which persists the entity via FlowerRepository and calls SaveChangesAsync(). On success, FlowerService calls IEventBus.Publish. The InMemoryEventBus iterates its handler list and invokes NotificationService.HandleAsync(). NotificationService constructs the email body and calls SmtpEmailSender.SendAsync(), which dispatches the message to the authenticated user's email address via Gmail (smtp.gmail.com, port 587) using App Password authentication. Control then returns to the controller, which redirects to the flower index page.

The export sequence shows: the user clicks Export JSON on the Flowers Index page, the browser sends GET /Export/Flowers?format=json with current sort and filter parameters. ExportController resolves the active IExportStrategy via ExportStrategyFactory, calls IFlowerService to retrieve the filtered and sorted flower list, maps results to FlowerExportDto, calls strategy.Export(), and returns a FileContentResult with the appropriate Content-Type and filename.

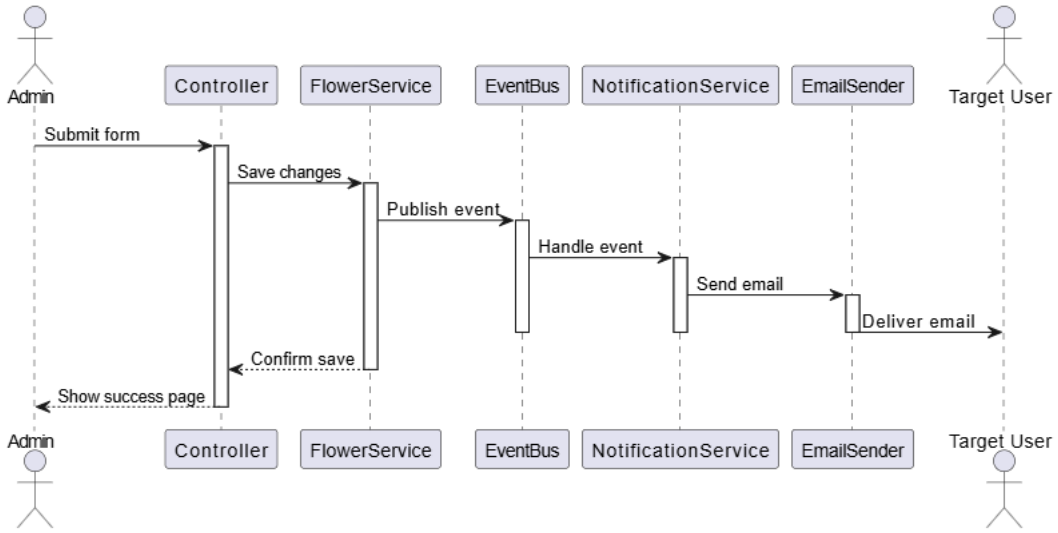


Figure 6: Sequence Diagram — Flower Create with Event Notification

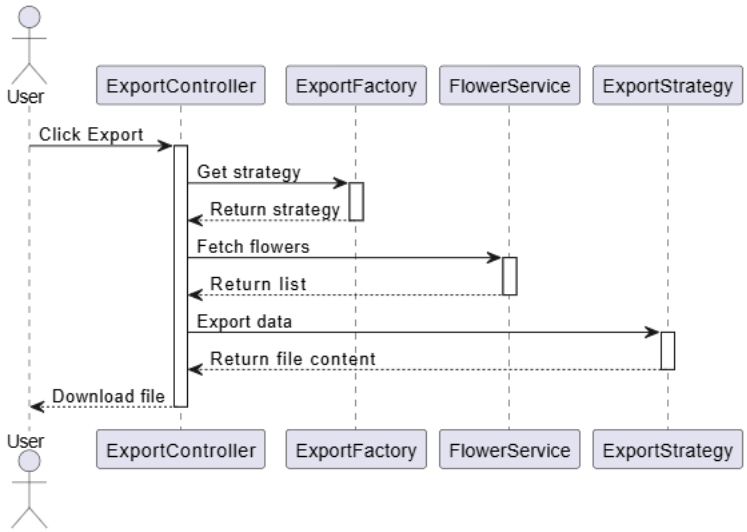


Figure 7: Sequence Diagram — Export Strategy

3 Implementation Phase - Application

3.1 Tools and Technologies

The application continues to use the same technology stack as Assignment 1, extended with the following:

- **Programming Language**: C# — Chosen for strong typing, excellent performance, and seamless integration with the .NET ecosystem
- **Database Management System**: MySQL 8.0 — Selected for open-source nature, reliability, and excellent Entity Framework Core support

- **Web Framework**: ASP.NET Core 8.0 — Provides high-performance HTTP pipeline, built-in dependency injection, and comprehensive MVC pattern implementation
- **ORM**: Entity Framework Core 8.0 — Enables type-safe database operations, automatic migrations, and LINQ query capabilities
- **Frontend**: Bootstrap 5 with Razor views — Ensures responsive design and professional appearance across devices
- **Authentication**: Cookie-based authentication with session management — Simple yet secure for web applications
- **Email**: System.Net.Mail.SmtpClient — Used in SmtpEmailSender for dispatching notification emails; configured via appsettings.json to use Gmail as the SMTP provider (smtp.gmail.com, port 587) with App Password authentication, enabling real email delivery to any recipient address without exposing account credentials
- **Serialization**: System.Text.Json (JSON export), System.Xml.Serialization (XML export), StringBuilder with reflection (CSV export) — No additional NuGet packages required for export functionality

3.2 Application Structure

The project extends the standard ASP.NET Core folder structure from Assignment 1 with two new top-level folders:

- **/Models**: Contains domain entities (Flower, Bouquet, Order, User, etc.) with validation attributes and navigation properties. Extended with FlowerExportDto for safe serialization during export.
- **/Data**: Houses AppDbContext class with Entity Framework configuration and database mappings. IDbContextFactory is now also registered here as a Singleton.
- **/Repositories**: Data access layer with interfaces and implementations for each entity type. Unchanged from Assignment 1.
- **/Services**: Business logic layer extended with IEmailSender, SmtpEmailSender, and NotificationService.
- **/Events**: New folder containing IDomainEvent, IEventHandler, IEventBus, InMemoryEventBus, FlowerCreatedEvent, FlowerUpdatedEvent, and FlowerDeletedEvent.
- **/Export**: New folder containing IExportStrategy, JsonExportStrategy, XmlExportStrategy, CsvExportStrategy, and ExportStrategyFactory.
- **/Controllers**: Extended with ExportController. FlowersController, CartController, OrderController, FloristController, and AdminController now carry explicit `[Authorize(Roles = "...")]` attributes.
- **/Views**: Flowers/Index.cshtml extended with an export toolbar containing Export JSON, Export XML, and Export CSV buttons that pass current filter and sort parameters.

- **/Middleware**: Custom middleware for session-based authentication. Unchanged from Assignment 1.
- **/Extensions**: Helper classes for session management and utility functions. Unchanged from Assignment 1.

3.3 User Manual

The application provides role-based interfaces tailored to different user types. The main navigation adapts based on user authentication status and role permissions.

Visitor Interface: Unauthenticated users can browse the flower catalog, view product details, and access login/registration pages. Visitors can export the full unfiltered flower list in JSON, XML, or CSV using the export toolbar on the Flowers page.

Client Interface: Authenticated customers gain access to shopping cart functionality, order placement, and personal order history. Clients can export the flower list with active filters and sorting applied.

Florist Interface: Florists see additional menu options for product management and order processing. They can view assigned orders, update order statuses, and export filtered flower lists.

Administrator Interface: Administrators have complete system access with user management, full order oversight, and system configuration options. When an Administrator creates, edits, or deletes a flower, an email notification is automatically sent to their registered email address confirming the action. The export toolbar is available on all flower management pages.

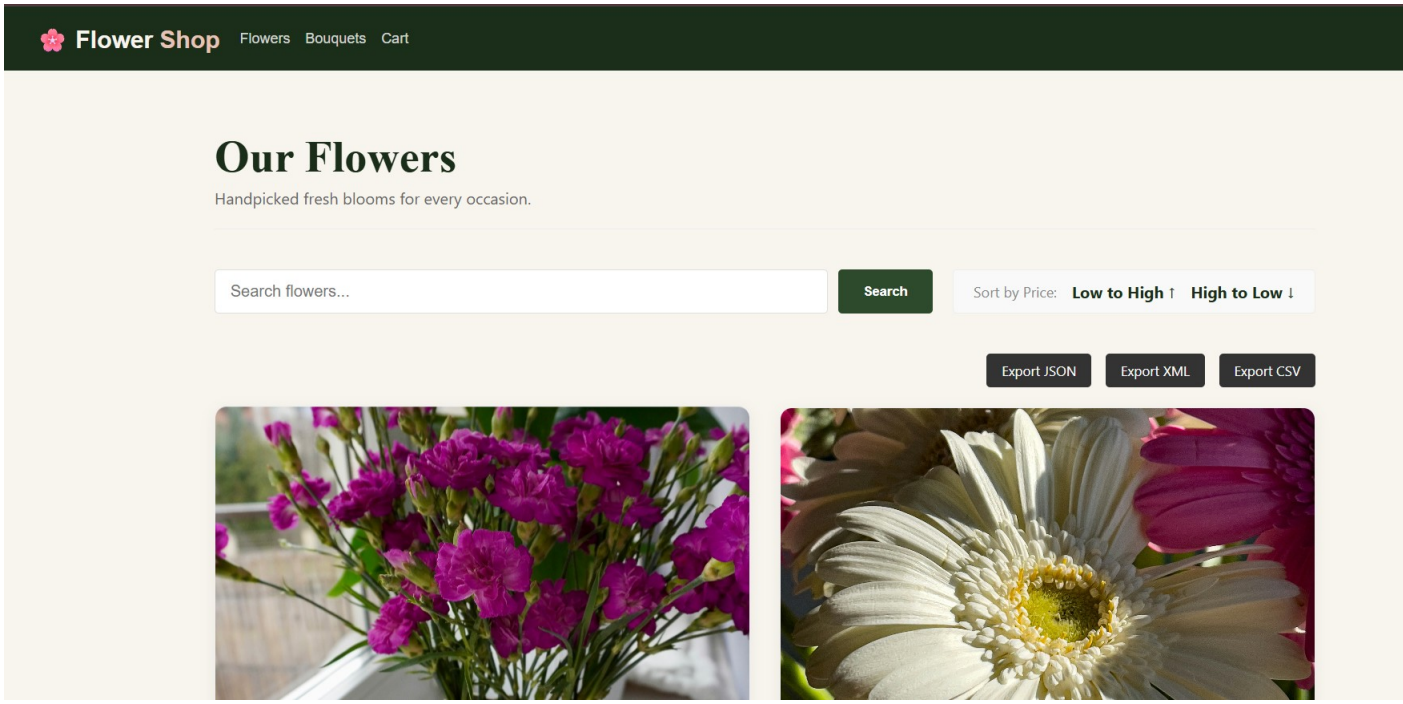


Figure 8: New View with Export Formats

4 Testing and Validation

The project extends the existing test suite from Assignment 1 with additional unit tests covering the new event-driven and export components. The framework stack remains **xUnit** combined with **Moq**, targeting .NET 8, with **Microsoft.EntityFrameworkCore 8.0.25** for transaction mocking.

New unit tests cover the following scenarios: FlowerService emits the correct domain event type and payload after CreateAsync, UpdateAsync, and DeleteAsync succeed; InMemoryEventBus invokes all subscribed handlers when Publish is called; NotificationService calls IEmailSender .SendAsync with the correct recipient address and a non-empty body for each of the three event types; ExportStrategyFactory returns the correct IExportStrategy implementation for the "json", "xml", and "csv" format strings; and each export strategy produces non-empty output for a non-empty input list.

IEmailSender is mocked in NotificationService tests to avoid real SMTP connections during the test run. IEventBus is mocked in FlowerService tests to assert that Publish is called with the expected event, without executing any handlers. This approach ensures all new components are tested in full isolation, consistent with the unit testing strategy established in Assignment 1.